

Vision-Language-Action (VLA) Agent for Visual Command Execution in ROS 2 Simulation

Information Technology Course
Module AIS
Prof. Dr. Peter Nauth

Saquib Attar
1567041
saquib.attar@stud.fra-uas.de

Muhammad Ahsan Ijaz
1566312
muhammad.ijaz@stud.fra-uas.de

Abstract—Getting robots to understand what we mean when we give them commands has been challenging, especially when those commands involve both what the robot sees and what we’re asking it to do. We developed a system that lets mobile robots take natural language instructions like “go to the red box” or “pass through the curtain to the blue object” and execute them in simulation. The system has three parts working together. A vision component identifies objects by color using HSV segmentation, tuned to recognize red at [0,70,50]-[10,255,255] and [170,70,50]-[180,255,255], purple at [125,70,80]-[155,255,255] and blue at [100,120,80]-[135,255,255]. Blue object detection is handled internally within the navigation policy module rather than through a standalone detector node. The language part uses Llama-3.3, a 70-billion parameter model through Groq’s API, which breaks down commands to figure out the requested action and key properties like whether an obstacle can be passed through. A policy component then matches what the language model extracted to what the vision system detected and sends navigation commands through ROS 2 and Nav2. We implemented two navigation approaches: exploration-based for directly reachable objects (red and purple boxes) where the robot searches until finding the target, and action-aware navigation for objects behind passable obstacles (blue box behind curtain), based on recent work on traversable obstacle handling. Tests on TurtleBot3 in Gazebo with three colored objects showed all commands parsed correctly, all thirty navigation attempts succeeded (ten per object), with completion times of 38-62 seconds depending on path complexity. People could phrase commands naturally without special syntax. Visual servoing achieved centering accuracy within 20 pixels. The key finding was successfully treating the curtain as traversable through policy-level safety bypass, letting the robot pass through to reach otherwise inaccessible objects.

Keywords—Vision-Language-Action (VLA), Large Language Models (LLM), natural language processing, robot navigation, HSV color segmentation, action-aware navigation, traversable obstacles, ROS 2, Nav2, visual servoing, human-robot interaction, Gazebo simulation, multimodal command understanding

I. INTRODUCTION

Robotic systems have become a lot more common in places where people are around, like hospitals, warehouses, and homes, especially over the last ten years. These service robots have to work alongside people in a way that feels simple and easy to understand. Most current robotic systems still need users to work with set command lists, special programming tools, or physical controls like buttons and joysticks [1]. This

makes it hard for most people to use, especially those who aren’t experts and just want to tell a robot what to do in plain language. The challenge is figuring out how to connect the way people naturally talk using language and visual cues with how robots understand and respond to the physical world.

Imagine when someone asks a service robot, “Can you bring me the red box which is placed on the left?” This simple request actually involves a bunch of complicated mental steps that people do without even thinking about it, but robots find really hard to handle. The robot needs to grasp the language used in the command, figure out which item in its view is the red box, work out what “on the left” means based on where it is, and then carry out the right movements and actions to get the job done. Traditional robotic systems usually handle vision, language understanding, and action execution as separate, independent parts [2], [3]. This split makes it hard to get the smooth connection which is needed for natural human-robot interaction.

Recent progress in artificial intelligence, especially with Large Language Models (LLMs) and Vision-Language Models (VLMs), has created new ways to tackle this challenge. LLMs have shown they can understand and generate natural language well, follow complicated instructions, and even think through different actions [4]. At the same time, VLMs have done well linking visual info with language, letting systems find objects just by using text queries [6], [7]. The coming together of these technologies has led to the rise of Vision-Language-Action (VLA) systems. These systems try to build a single framework where robots can understand commands that use different types of input and then turn those commands into real-world actions [8]-[10].

Even with these improvements, there are still some basic challenges that haven’t been solved. First, existing VLA systems usually need a lot of training data that’s tailored to the specific robot and the task it has to do [8]. Second, many methods face a hard time dealing with how flexible natural language is. Generally people can say the same thing in so many ways, like “go to the red cube,” “move toward the red

box,” or “navigate to the red object.” Systems need to be able to handle all these different ways of saying the same thing without missing a beat. Third, and especially important for what we do, typical navigation systems see every obstacle they find as something to get around. The viability of this concept diminishes when confronted with practical scenarios where certain impediments are surmountable. Consider, for instance, a hospital delivery robot’s requirement to navigate through curtains to access patient rooms, or a lawn-maintenance robot’s need to traverse grass rather than perceiving it as an obstruction [11].

Zhang et al. recently tackled this traversable obstacle problem by introducing the idea of “action-aware attributes” [11]. Their system uses an LLM to understand natural language commands and pull out not only the target object but also details about whether obstacles need to be avoided or if they can be passed through. For instance, the command “pass through the curtain to deliver medicine” shows that the curtain can be passed through. They use a VLM called Grounding DINO to find landmarks in images and then break down LiDAR point clouds into areas that can be traveled through and areas that can’t. This helps build a costmap that takes actions into account for planning the path. Their approach gets good results, but it needs a lot of complicated work like setting up VLM, calibrating the LiDAR and camera, segmenting point clouds, and making custom Nav2 plugins. This complexity makes it hard for researchers and practitioners to explore VLA concepts without having access to a lot of infrastructure.

In this work, we introduce a VLA integration system that helps mobile robots understand natural language commands and carry out matching navigation tasks in simulation. Our system tackles the main challenges of multimodal command understanding while keeping things simple enough for quick prototyping and teaching purposes. We combine three main parts: a vision module that uses HSV color segmentation to detect objects, and a language module that runs on Llama-3.3 LLM with 70 billion parameters each are accessed through Groq’s API to handle command parsing and extract action-related attributes. There’s also a policy module that connects language descriptions to the objects detected and creates navigation commands using ROS 2 and the Nav2 framework.

We use two different ways to navigate depending on the situation. For objects that the robot can see directly in the environment, like a red box or a purple cube out in the open, it starts by exploring randomly. Once the robot spots the target object through its camera, it then switches to using coordinates and visual servoing to precisely center on it. For objects hidden behind obstacles you can go through, like a blue box inside a room behind a curtain, we use action-aware navigation based on Zhang et al.’s work [11]. Instead of relying on VLM integration and LiDAR segmentation, we use known object coordinates and mimic the action-aware

costmap behavior by skipping safety checks only when the robot is close to obstacles it can pass through. This simplified approach achieves similar high-level behavior where the robot successfully passes through curtains to reach otherwise inaccessible objects while being more accessible for educational and research prototyping.

II. RELATED WORK

Over the past few years, combining vision, language, and action in robotic systems has become an important focus in research. Our work builds on and adds to several main areas of research, which we’ve grouped into four themes: vision-language models for robotics, large language models used for robot control, ways to navigate through obstacles that can be crossed, and visual servoing methods. This section looks over the key work in each area and shows how our approach fills in the gaps that are still there.

A. Vision-Language Models for Robotics

The link between visual perception and language understanding has been a challenge in robotics for a long time. Early systems used hand-crafted features and had a limited vocabulary, which made them fragile and hard to expand. Deep learning changed things a lot, especially with models like CLIP [6] showing that vision and language can be linked together in the same space by using contrastive learning on a huge amount of web data. This breakthrough made it possible to recognize objects from text descriptions without any prior examples, opening up new ways for robots to be more adaptable.

Starting from this point, researchers began looking into how to turn vision-language understanding into actual robotic movements. Brohan et al. introduced RT-2 [8], a model that builds on vision-language systems by allowing them to generate robot actions directly. They use pre-trained vision-language models and then fine-tune them with robotic demonstration data, so the system can carry out manipulation tasks just by following natural language instructions. RT-2 showed it could handle new tasks well, even when they were described in ways it hadn’t seen during training. The system needs a lot of robot-specific training data and mainly concentrates on manipulation instead of navigation tasks.

Recently, Kim et al. created OpenVLA, an open-source vision-language-action model that helps make these abilities easier for researchers to use [9]. OpenVLA offers pre-trained models and training pipelines that you can adjust to work with different robotic platforms. This is a very big step toward making VLA research more accessible, but still the models need a lot of computing power to train and run. Also, they mainly focus on tabletop manipulation tasks, not on mobile robot navigation.

Cai et al. suggested a way to create robot action sequences by using vision-language models combined with visual prompts [10]. The method which they used depends on visual markers in the environment to steer the vision-language model's attention, which makes it easier for the model to detect relevant objects and come up with the right action sequences. This work demonstrates how visual prompts can help with grounding, but it still relies on having those prompts around, which might not always be practical in real-world situations.

A common problem with these VLA methods is that they treat the environment as if it never changes, assuming every obstacle they detect is something to avoid. This approach is effective in many situations, yet it encounters difficulties when robots must navigate or interact with flexible objects. Think curtains, grass, or anything else that can be walked on or pushed.

B. Large Language Models for Robot Control

Large Language Models have shown they can really understand and generate natural language, which has made researchers interested in trying them out with robotic systems. One main benefit of LLMs is that they can understand complicated, naturally worded instructions without needing specialized training data or strict command formats [4], [5].

Several recent works have investigated using LLMs as high-level planners for robotic tasks. These systems typically use the LLM to break down natural language instructions into a sequence of discrete actions, which are then executed by lower-level controllers [12], [13]. For example, an instruction such as "clean the table and put the dishes in the sink" could be broken down into several steps: find the table, go to the table, pick up the dish, find the sink, go to the sink, place the dish, and repeat. The large language model's ability to use common sense helps it understand the implied order of these actions and deal with different ways the task could be described.

Conversely, a significant constraint inherent in large language models (LLMs) is their exclusive emphasis on linguistic data. Although these models can analyze actions and objects referenced within textual input, they lack the capacity for direct world perception or the ability to independently validate the existence and spatial location of the entities they describe. This creates a grounding problem that the LLM might generate a perfectly reasonable plan in abstract terms, but that plan may not be executable in the actual environment the robot finds itself in.

C. Interactive Navigation with Traversable Obstacles

Traditional mobile robot navigation systems are built on the basic idea that all detected obstacles should be avoided. This

works well in structured environments with clearly defined free space, but becomes problematic in real-world scenarios where some obstacles are actually traversable [14], [15]. A delivery robot in a hospital should be able to pass through privacy curtains to reach patient rooms, and an agricultural robot needs to navigate through crop rows rather than treating plants as impassable barriers.

Zhang et al. recently tackled this problem by introducing the concept of "action-aware attributes" for robot navigation [11]. Their system uses a Large Language Model (GPT-3.5) to parse natural language commands and extract not just the target object but also attributes indicating how the robot should treat different elements in the environment. For example, the command "pass through the curtain to deliver the medicine" indicates that the curtain should be treated as traversable rather than as a barrier. They combine this with Grounding DINO, a vision-language model, to detect the mentioned landmarks in camera images and obtain their bounding boxes.

The critical innovation in their work is the construction of an action-aware costmap. They transform LiDAR point clouds to the camera coordinate frame, then use the bounding boxes from the vision-language model to segment the point cloud into regions corresponding to different landmarks. Based on the action-aware attributes extracted by the LLM, they assign different costs to these regions such that traversable obstacles get cost 0 (free space), while untraversable obstacles get high cost values. This modified costmap is then used by the A* path planner to find routes that may pass through traversable obstacles.

Although their method shows impressive results in both simulations and real-world tests with a quadruped robot moving through curtains and grass, it requires several complex components. First, accurate camera-LiDAR calibration is essential for projecting point clouds onto the image plane. Second, deploying and running the Grounding DINO vision-language model adds computational overhead. Third, the point cloud segmentation must handle cases where bounding boxes overlap or where parts of traversable and untraversable objects appear in the same region. Fourth, they implement a custom Nav2 plugin to handle the action-aware costmap layers.

D. Visual Servoing and Object Detection

Visual servoing, a technique that employs visual feedback to regulate robotic motion, has been the subject of considerable research in the domains of manipulation and navigation [16], [17]. Image-based visual servoing (IBVS) methods derive control commands directly from image features, circumventing the need for explicit 3D reconstruction; this characteristic renders them resilient to calibration inaccuracies

and computationally efficient.

In object detection, using color-based segmentation in the HSV color space has proven effective for identifying objects with distinct colors, especially in controlled environments [18]. The HSV color space, which separates color information (hue) from brightness (value), offers some robustness to changes in lighting, a feature that distinguishes it from the RGB color model. Nevertheless, the precise calibration of HSV ranges is of paramount importance; consequently, this method may encounter difficulties in intricate lighting scenarios or when the object’s coloration closely resembles that of the background.

The ROS 2 Navigation Stack (Nav2) provides a robust framework for mobile robot navigation, handling costmap creation, path planning, and trajectory following [19]. Nav2’s layered costmap architecture allows different sensor inputs and semantic information to be combined, though integrating action-aware attributes requires either custom plugins or creative use of existing layers.

E. Summary and Research Gap

The current body of research has made considerable strides in vision-language comprehension, natural language command parsing, and adaptive navigation techniques. Nevertheless, a deficiency persists in the application of these sophisticated functionalities to educational contexts and the rapid prototyping of novel applications. Zhang et al.’s work [11] shows what is possible with full VLA infrastructure, but the complexity of deploying vision-language models, calibrating multiple sensors, and implementing custom navigation plugins creates barriers for researchers and students who want to explore these concepts.

This work addresses the identified gap through a simplified VLA architecture that maintains core action-aware navigation behavior while significantly reducing infrastructure requirements, as described in Section III.

III. SYSTEM ARCHITECTURE

This section describes the architecture of our Vision-Language-Action integration system for robotic navigation. The system’s architecture is built on three main modules: a vision module that identifies objects, a language understanding module that interprets commands, and a policy module that handles navigation and control. These modules communicate through ROS 2 topics and services, creating an integrated pipeline from human language input to physical robot motion.

Figure 1 shows the overall system architecture. When a user provides a natural language command such as “go to the red box” or “pass through the curtain to find the blue object,” the language understanding module parses this command and extracts action-aware attributes including the target object,

required actions, and properties of environmental elements (such as whether they are traversable). At the same time, the vision module processes camera images to find and locate colored objects in the environment. The policy module then grounds the linguistic descriptions to the detected objects, determines the appropriate navigation strategy, and executes the corresponding motion commands through the Nav2 navigation framework and visual servoing controllers.

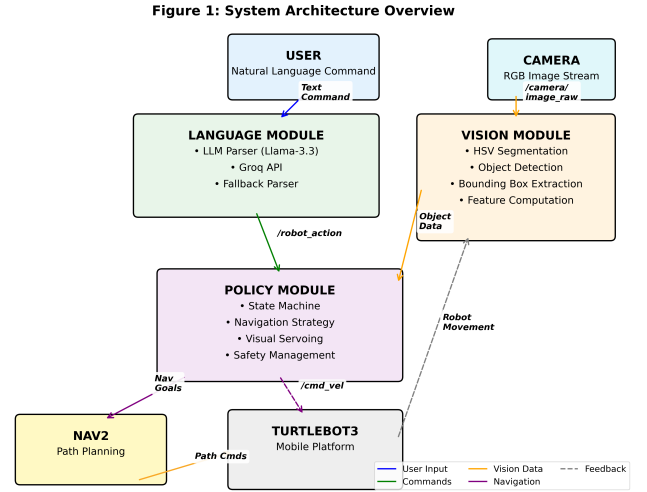


Fig. 1. System Architecture Overview

A. System Overview and Data Flow

The system works like a pipeline, using both parallel and sequential processing steps. The vision module operates continuously, analyzing camera images at a rate of roughly 30 Hz to identify objects by their color signatures. Consequently, this module furnishes real-time awareness of the objects present and their locations within the robot’s field of view. The language understanding module activates when the user provides a command, parsing the natural language input through either an LLM-based approach (using Llama-3.3 via Groq’s API) or a fallback rule-based parser if the LLM is unavailable or returns unclear results.

The policy module serves as the primary coordinator, subscribing to both the command topic, which transmits parsed action commands such as **START_RED** or **START_BLUE_THROUGH_CURTAIN**, and the vision topic, which supplies object bearing and size data. Depending on the command type, the policy module then selects one of two navigation strategies: exploration-based navigation for directly accessible objects, or action-aware navigation for objects situated behind traversable obstacles. It then coordinates with Nav2 for path planning and obstacle avoidance while also implementing visual servoing for precise final approach to detected objects.

Modules communicate with each other using standard ROS 2 topics. The language module sends messages to `/robot_action`, which are String messages with command identifiers. The vision module sends bearing values to `/red_object/bearing` and `/purple_object/bearing` (`std_msgs/Float32`) for red and purple objects, respectively. At the same time, the policy module gets information from `/camera/image_raw/compressed` (`sensor_msgs/CompressedImage`) for height-based visual servoing and blue object detection. The policy module concurrently dispatches velocity commands to `/cmd_vel`, utilizing the `geometry_msgs/Twist` message type. This occurs in conjunction with Nav2’s action server, which handles navigation objectives. This architectural style, which is characterized by loosely connected components, allows for the independent development, testing, and modification of individual modules, while still maintaining overall system integration.

B. Vision Module

The vision module mainly identifies objects and extracts visual features. This is done by segmenting the image based on color in the HSV color space. The decision to utilize HSV rather than RGB stemmed from its inherent ability to decouple color data, specifically hue, from brightness, or value. This segregation enhances the dependability of detection, particularly in fluctuating illumination conditions, a scenario frequently encountered within simulated settings. The module gets data from the robot’s camera, and then it processes each image through a series of steps.

The camera’s BGR image is first converted to the HSV color space using OpenCV. Then, thresholding is used to create binary masks for each color of interest. Because red objects cover a wide range of hues, two masks are created for them. These masks correspond to hue ranges of $[0^\circ, 10^\circ]$ and $[170^\circ, 180^\circ]$, and they are combined using a bitwise OR operation. In contrast, a single mask is used for purple objects, covering the hue range of $[125^\circ, 155^\circ]$. Blue objects, specifically the target behind the curtain, are identified using a hue range of $[100^\circ, 135^\circ]$. Furthermore, each color mask incorporates saturation and value constraints to eliminate low-saturation (grayish) regions, as well as very dark or bright areas, which are likely indicative of noise or lighting artifacts.

After creating the binary masks, we use OpenCV’s `findContours` function to find the contours. We use the `RETR_EXTERNAL` flag to only find the outer boundaries. We filter these contours by area, discarding any with contour area smaller than 400 pixels within the navigation policy module to eliminate noise during visual servoing and target approach. The independent detector node uses a 200-pixel threshold for estimating direction, which allows for earlier detection at greater distances. If multiple contours are still

present after filtering, the largest one is chosen as the main object of that color. From this contour, a rectangle is created, and its center point (cx , cy), width (w), and height (h) are calculated.

The bounding box provides two important pieces of information that are crucial for navigation. First, the horizontal center position, cx , shows the object’s angular offset from the robot’s forward direction. We compute the pixel error as cx minus the image center, then convert this to an angular bearing in radians. This bearing information is published on object-specific topics (`/red_object/bearing`, `/purple_object/bearing`) for red and purple objects. The bounding box height, denoted as h , is calculated within the camera callback of the navigation policy module. This height serves a dual purpose: it triggers visual servoing and indicates whether the operation was successful. Unlike some data, it’s not published as a distinct topic. Blue object detection is also handled internally within the navigation policy module using HSV range $[100, 135]$ with a tighter saturation constraint ($S_{\min} = 120$) to minimize false positives in the enclosed room environment, activating exclusively upon receipt of the `START_BLUE_THROUGH_CURTAIN` command. Second, the bounding box height h provides a measure of the object’s apparent size in the image, which correlates with distance when larger objects are closer. The policy module uses this height information to determine when to switch from Nav2 navigation to visual servoing.

A key feature of our vision module is its ability to work continuously, regardless of the robot’s navigation state. Consequently, the vision system maintains a constant awareness of its surroundings, even when the robot isn’t actively searching for objects. This design choice aligns with the exploration-based navigation strategy. In this strategy, the robot initially doesn’t know where objects are located and instead detects them opportunistically during periods of random exploration.

C. Language Understanding Module

The language understanding module’s function is to translate natural language commands into specific action identifiers, which the policy module then executes. This module utilizes a two-tiered approach: a primary LLM-based parser, engineered to handle the inherent flexibility and nuance of natural language, and a rule-based fallback parser, which ensures system reliability in situations where the LLM is either unavailable or uncertain.

The LLM-based parser uses Llama-3.3, a language model with 70 billion parameters, which is accessed through Groq’s cloud API. We chose this approach rather than running a local model because Groq’s infrastructure provides extremely fast inference (typically under 500 milliseconds per query) without requiring expensive local GPU resources. The model

processes commands through a carefully designed system prompt that instructs it to extract action-aware attributes from natural language and map them to specific command identifiers.

The system prompt is a key player in how well a language model works. It lays out the specific command types – `START_RED`, `START_PURPLE`, `START_BLUE_THROUGH_CURTAIN`, and `UNKNOWN`, and offers examples of how various wordings should translate into those commands. Critically, it instructs the model to identify not just target objects but also their traversability properties. For instance, the phrase “pass through the curtain” indicates that the curtain should be treated as traversable ($P_a = 1$ in Zhang et al.’s notation [11]), while “avoid the curtain” would indicate it should be treated as a barrier ($P_a = 0$).

When a command is processed, the language module sends the user’s input and the system prompt to the Groq API. The API subsequently sends back a text response. We then sift through this response to pinpoint the command identifier. Should the response unmistakably contain one of our established commands, such as “`START_BLUE_THROUGH_CURTAIN`,” we proceed to execute that command. If the response is unclear or the language model hesitates, the system will use the rule-based parser by default.

The fallback parser employs a more straightforward yet readily understandable approach, predicated on the identification of keywords and the application of logical principles. It examines the input text for action keywords (e.g., go, move, navigate, find, and pass through), color keywords (such as red, purple, and blue), and obstacle keywords (curtain and door). Subsequently, the system employs decision rules to determine the correct command. For example, if the system identifies the terms “blue” and “curtain,” alongside a movement verb such as “pass through,” it will then issue the command `START_BLUE_THROUGH_CURTAIN`. The fallback mechanism proved beneficial throughout the development process, guaranteeing the system’s persistent functionality. This was particularly crucial in scenarios where the Groq API was inaccessible or when network latency rendered LLM queries unfeasible.

The two-layer architecture serves two primary functions. First, it offers adaptability, enabling the LLM to accommodate novel linguistic constructions and evolving language patterns that would pose challenges for a strictly rule-based system. Secondly, it ensures the dependability of the rule-based parser, thereby guaranteeing consistent functionality for established command formats. During our testing, the LLM successfully processed all standard commands and most variations. In contrast, the fallback parser achieved perfect accuracy on our predefined test set.

D. Policy and Navigation Module

The policy module acts as the main decision-making part, controlling navigation based on the commands it receives and the objects it detects. This module uses two different navigation strategies, depending on whether the target object is directly reachable or if it’s behind a movable obstacle. Figure 2 shows the state machine used for exploration-based navigation, which is used for RED and PURPLE objects.

Figure 2: State Machine - Exploration-Based Navigation

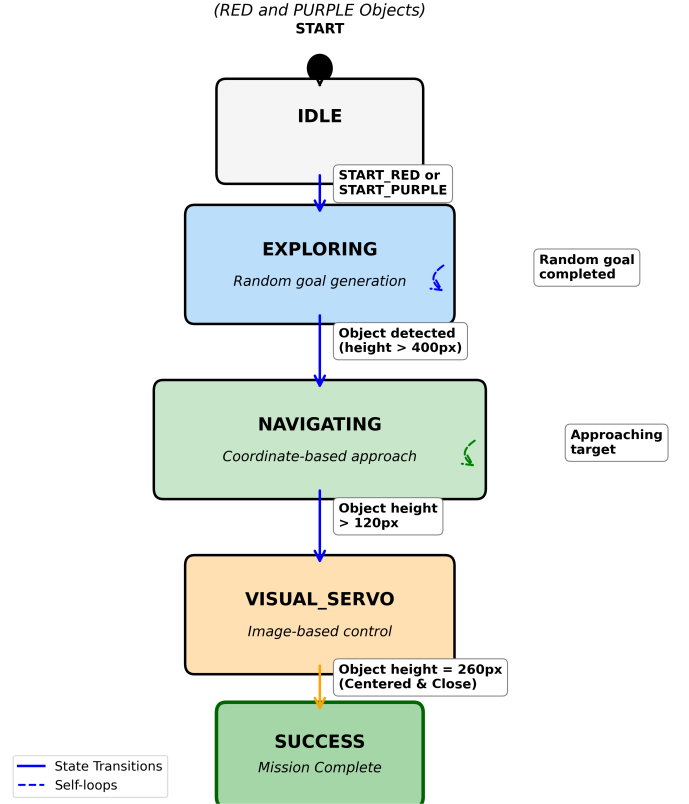


Fig. 2. State Machine

For navigation based on exploration, used for RED and PURPLE objects, the system uses a state machine with five main states: IDLE, EXPLORING, NAVIGATING, VISUAL SERVO and SUCCESS. Upon receiving a `START_RED` or `START_PURPLE` command while idle, the policy module shifts into the EXPLORING state. It then begins dispatching random navigation goals to Nav2. These goals are picked at random from a range of $[-8.0, 8.0]$ meters for both the x and y coordinates. This approach encourages the system to thoroughly investigate its surroundings.

During exploration, the vision module constantly processes images from the camera. When an object is detected that matches the target color and is larger than a set minimum size (indicating it’s close enough to be relevant), the vision module’s detection triggers a state change, moving from

EXPLORING to NAVIGATING. At this point, the robot cancels the current random goal and switches to a coordinate-based approach strategy.

In the NAVIGATING state, the robot knows the approximate location of the target object, based on our predefined coordinate map. In a real-world system, these coordinates might come from a previous mapping phase or a database of object locations. The robot subsequently relays minor navigation objectives to Nav2, the system that assists in its approach to the pre-set coordinates. The robot's step size is adaptive, adjusting as it approaches its destination. Initially, the robot employs larger steps, each measuring 1.3 meters, when situated more than 4 meters from the target. As it moves closer, the step size decreases to a medium 0.9 meters, applicable within the 2 to 4 meters range. Ultimately, when within 2 meters of the target, the robot adopts smaller steps, each 0.5 meters in length. This prevents overshoot while maintaining reasonable progress rate.

For action-aware navigation (used for the BLUE object behind the curtain), the policy module implements a different strategy inspired by Zhang et al.'s work [11]. Figure 3 illustrates this two-phase navigation approach in the actual Gazebo simulation environment. When it receives a `START_BLUE_THROUGH_CURTAIN` command, it recognizes that this involves a two-phase navigation task: first reaching the traversable obstacle (curtain), then continuing to the final goal (blue object).

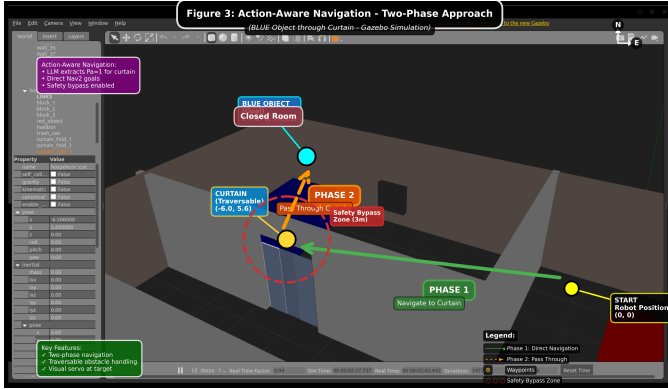


Fig. 3. Two Phase Navigation Hybrid

The module first marks "CURTAIN" in its internal set of traversable obstacles. It then sets CURTAIN as the current navigation target and BLUE as the final target, and transitions directly to NAVIGATING state (skipping exploration since we know where the curtain is located). The robot navigates toward the curtain coordinates using direct Nav2 goals rather than incremental steps since we don't need to be as precise in approaching the curtain, we simply command Nav2 to navigate to the curtain position.

The critical difference in action-aware navigation is how safety checks are handled. Normally, the robot's safety system would detect the curtain (or at least the walls of the closed room) through LiDAR and either stop or plan a path around the obstacle. However, for traversable obstacles, we want the robot to pass through. Rather than implementing a custom Nav2 plugin that modifies the costmap (as Zhang et al. did), we simulate this behavior at a higher level by bypassing safety checks when the robot is within 3 meters of any obstacle marked as traversable. This allows the robot to continue moving toward and through the curtain entrance.

When the robot comes within 1.2 meters of the curtain coordinates, the policy module switches the target from CURTAIN to BLUE and continues navigation toward the blue object coordinates. At this point, the robot is inside the closed room (having passed through the curtain entrance), and navigation proceeds normally toward the blue object. When the robot gets within 2.0 meters of the blue object (farther than the 1.2 meters used for red and purple), it stops Nav2 navigation and waits for the vision module to detect the blue object in the camera. Once detected and exceeding the visual servo trigger threshold, the system transitions to VISUAL_SERVO state and performs the same image-based centering and approach as for the other objects.

This two-phase approach with safety bypass achieves similar behavior to Zhang et al.'s action-aware costmap [11] but with much simpler implementation. The robot successfully navigates through the curtain to reach an object it could not access otherwise, demonstrating the core concept that linguistic cues in commands can inform navigation behavior. The key simplification is that rather than modifying the actual costmap used by the path planner, we handle traversability at the policy level through selective safety check bypass and known coordinates.

For both navigation strategies, the final approach to the target object uses visual servoing for precise positioning. Figure 4 shows the detailed control flow for this visual servoing process. As the robot approaches and the object in the camera view gets larger, it eventually exceeds the visual servo trigger threshold, which is set at 120 pixels in height. This causes the system to switch to the VISUAL_SERVO state. Nav2 navigation is cancelled and the robot switches to direct velocity control based on image features. The visual servo controller computes angular velocity to center the object horizontally in the image (minimizing the pixel error between object center and image center) and forward velocity to approach the object until it reaches the target size (260 pixels in height). Once this target size is achieved, the robot stops and the mission is complete.

Figure 4: Visual Servoing Control Flow

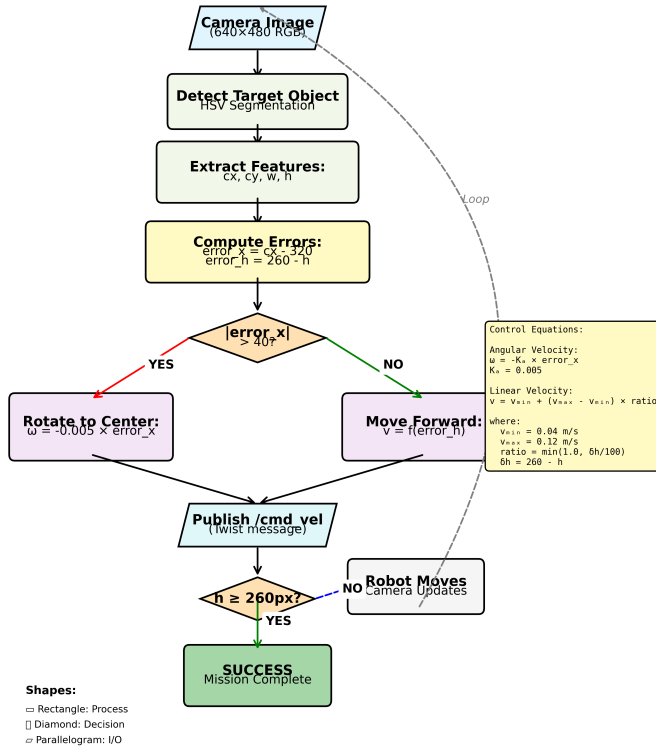


Fig. 4. Visual Servo

E. Integration and Robustness Considerations

Design decisions play a significant role in enhancing the system's resilience and usability. First, limiting the rate of navigation goal commands, which requires a minimum of 0.5 seconds between goals, prevents the robot from sending too many commands to Nav2. This could cause instability or lead to the rejection of goals. Second, the tracking of frames without object detection allows the system to handle temporary occlusions, if an object disappears briefly (perhaps behind a post or other obstacle), the robot continues its approach rather than immediately abandoning the task.

Furthermore, the modular design's separation of functions reduces the chance of a system-wide failure when a single component has a problem. For example, if the vision module fails to identify objects, the navigation system can still work by using pre-stored coordinates. Similarly, if the LLM parser has an error, the backup parser can still provide basic functions. This approach, which allows for graceful degradation, ultimately boosts the system's reliability.

Consequently, the state machine's design, which is based on clear transitions, guarantees predictable operational behavior. Each state is controlled by clearly defined entry and exit conditions. This streamlines the debugging process, allowing

us to better grasp the robot's behavior during each test. This was very helpful during the development and testing phases.

IV. IMPLEMENTATION DETAILS

This section describes the technical implementation of our Vision-Language-Action integration system. It encompasses hardware and software specifications, along with parameter settings and the specifics of the algorithms. To support reproducibility, we provide thorough tables of key parameters and pseudocode for the main algorithms.

A. Hardware and Software Platform

The Gazebo simulation environment was utilized for the implementation and testing of our system, specifically employing the TurtleBot3 Waffle Pi mobile robot platform. Simulation was selected for the preliminary development and testing phases, primarily because of its benefits in expediting prototyping, facilitating parameter adjustment, and ensuring experimental consistency. Furthermore, the simulation environment offers realistic physics modeling, encompassing collision detection, sensor noise, and dynamic constraints.

The software stack is constructed using ROS 2 Humble, which operates on Ubuntu 22.04 LTS. All node implementations are written in Python 3.10. We utilize several libraries: OpenCV 4.7 for computer vision tasks, NumPy for numerical calculations, and the rclpy client library to facilitate communication within ROS 2. Navigation capabilities are provided by the Nav2 framework, which handles costmap construction, global and local path planning, and trajectory following. A 2D occupancy grid map of the hospital simulation environment was first constructed by teleoperating the TurtleBot3 Waffle Pi through the environment using the TurtleBot3 Cartographer SLAM package. The resulting map was saved as a PGM image file and subsequently loaded in all navigation experiments via the Nav2 localization stack, which employs AMCL (Adaptive Monte Carlo Localization) for real-time robot pose estimation within the map frame. During the development phase, RViz2 was employed to provide real-time visualization of the map, the robot's position, sensor data, and navigation objectives. This facilitated parameter adjustments and system verification. For natural language processing, we access the Llama-3.3 70B model through Groq's cloud API with a typical response time under 500 milliseconds.

The TurtleBot3 Waffle Pi model in Gazebo includes a 360-degree LDS-01 LiDAR sensor with 1-degree angular resolution and a forward-facing RGB camera with 640x480 resolution at 30 Hz. The robot's differential drive system allows it to reach a maximum linear speed of 0.22 m/s and a maximum angular speed of 2.84 rad/s. These specifications match the physical TurtleBot3 hardware, which makes it

easier to use the algorithms developed on real robots.

B. Environment and Object Configuration

Table I provides the coordinates for all objects and landmarks within the simulated environment, referenced within the map frame, which is centered at the environment's origin. The RED and PURPLE objects are situated in open, readily accessible zones to facilitate the evaluation of exploration-based navigation strategies. Conversely, the BLUE object is situated within a confined space, with the CURTAIN demarcating the point of entry; this arrangement was deliberately designed to evaluate action-aware navigation skills when negotiating obstacles. These placements were selected to offer adequate spacing (a minimum of 7 meters between any two objects) to facilitate clear differentiation during navigation exercises.

TABLE I
OBJECT AND LANDMARK COORDINATES IN MAP FRAME

Object	x (m)	y (m)	Description
RED	0.607	7.538	Open accessible area
PURPLE	7.561	-0.768	Open accessible area
BLUE	-7.076	7.815	Inside closed room
CURTAIN	-6.0	5.6	Room entrance (traversable)

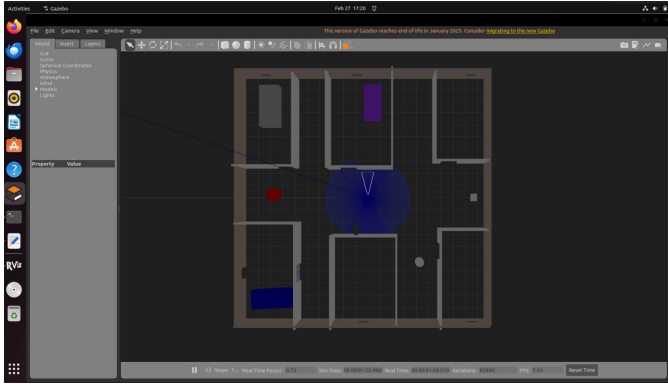


Fig. 5. Gazebo simulation environment showing layout with the three target objects: red object (left), purple object (top center), and blue object (center room) with curtain at the entrance. The TurtleBot3 Waffle Pi robot is visible at the center of the environment.

C. Vision System Parameters

The vision system uses HSV color space for object detection due to its robustness to lighting variations. Table II presents the carefully tuned HSV ranges for each target color. For red objects, which are found around the hue circle (where 0° and 360° represent the same color), we use two different ranges combined with a logical OR operation. Saturation and value constraints are then applied to remove desaturated (gray) and very dark or bright areas. These areas usually represent noise or lighting problems, rather than the actual

TABLE II
HSV COLOR RANGE PARAMETERS FOR OBJECT DETECTION

Color	H _{min}	H _{max}	S _{min}	S _{max}	V _{min}	V _{max}
RED (range 1)	0	10	70	255	50	255
RED (range 2)	170	180	70	255	50	255
PURPLE	125	155	70	255	80	255
BLUE	100	135	120	255	80	255

colored objects.

These ranges were determined through iterative testing in the Gazebo environment. We found that slightly broader hue ranges improved detection reliability under the simulated lighting conditions, while tighter saturation constraints reduced false positives from background elements. The navigation policy module applies a contour area threshold of 400 pixels for visual servo triggering and target approach. The standalone detector node uses a threshold of 200 pixels for bearing estimation. Both thresholds were validated across the maximum exploration range of 8 meters. Blue object detection is now integrated into the navigation policy module, as opposed to being a separate detector node. This approach employs a more stringent saturation minimum of 120, a significant increase from the 70 used for red and purple. The goal is to minimize false positives caused by background clutter in the confined space of the room. It is activated exclusively upon receipt of the START_BLUE_THROUGH_CURTAIN command during action-aware navigation.

D. Navigation and Control Parameters

Table III summarizes all navigation and control parameters used in our system. The exploration range of ± 8.0 meters provides full coverage of the 16×16 meter simulation environment. The adaptive step size creates smooth transitions between long-range approach and precise positioning, the system takes large 1.3 meter steps when far from the target (distance > 4 meters), intermediate 0.9 meter steps at mid-range (2-4 meters), and smaller 0.5 meter steps when close (distance < 2 meters), preventing overshoot while maintaining efficiency.

The visual servo trigger threshold of 120 pixels corresponds to approximately 1.5-2 meters from the object, at which point image-based control becomes more reliable than odometry-based navigation. A target height of 260 pixels corresponds to roughly 0.5 meters from the object, thereby facilitating unobstructed camera perspectives suitable for prospective manipulation operations. Furthermore, a central tolerance of ± 40 pixels, which is equivalent to $\pm 6.25\%$ of the image width, is implemented to guarantee sufficient alignment, while simultaneously accommodating inherent real-world variabilities.

TABLE III
NAVIGATION AND VISUAL SERVOING CONTROL PARAMETERS

Parameter	Value	Description
<i>Exploration Parameters</i>		
Random exploration range	± 8.0 m	Uniform sampling bounds
Goal timeout	30 s	Maximum time per goal
<i>Navigation Parameters</i>		
Adaptive step (far)	1.3 m	Distance > 4 m from target
Adaptive step (mid)	0.9 m	Distance 2–4m from target
Adaptive step (near)	0.5 m	Distance < 2 m from target
Stop distance (RED/PURPLE)	1.2 m	Nav2 stop threshold
Stop distance (BLUE)	2.0 m	Nav2 stop threshold
Safety bypass radius	3.0 m	Around traversable obstacles
Rate limit	0.5 s	Min. time between goals
<i>Visual Servo Parameters</i>		
Trigger threshold	120 px	Object height to activate
Target height	260 px	Success criterion
Center tolerance	± 40 px	Horizontal error bound
Angular gain (K_a)	0.005	Rotation control gain
Min linear speed	0.04 m/s	Forward velocity lower bound
Max linear speed	0.12 m/s	Forward velocity upper bound

To facilitate smooth rotation and reduce oscillation, the angular gain was set to 0.005. The linear speed range, spanning from 0.04 to 0.12 m/s, was selected to optimize the balance between approach time, safety, and control stability. For the BLUE object, a more extended stop distance of 2.0 meters was implemented, as opposed to the 1.2 meters utilized for the RED and PURPLE objects. This adjustment ensured the robot ceased Nav2 navigation and switched to visual servoing before entering the curtain area, thereby improving the dependability of the traversal.

E. ROS 2 System Architecture

Table IV shows the ROS 2 topics and message types used for communication between nodes. The system uses a publish-subscribe model. In this model, the vision module constantly publishes detection results. The language module publishes parsed commands, and the policy module subscribes to both of these while also publishing control commands. This loosely coupled design allows independent development and testing of each module.

TABLE IV
ROS 2 TOPICS AND MESSAGE TYPES

Topic Name	Message Type	Pub/Sub
/camera/image_raw/ compressed	sensor_msgs/ CompressedImage	Camera / Policy
/scan	sensor_msgs/ LaserScan	LiDAR / Policy
/robot_action	std_msgs/String	Language / Policy
/red_object/ bearing	std_msgs/Float32	Vision / Policy
/purple_object/ bearing	std_msgs/Float32	Vision / Policy
/cmd_vel	geometry_msgs/ Twist	Policy / Robot
/odom	nav_msgs/ Odometry	Robot / Policy
/tf	tf2_msgs/ TFMessage	Robot / All nodes

The Nav2 action server is accessed through the navigate_to_pose action interface (nav2_msgs/NavigateToPose), which allows the policy module to send goal poses and receive feedback on navigation progress. The TF2 transform system maintains the coordinate frame relationships, with the policy module using transforms between the map frame and base_link frame to determine the robot’s current position.

F. Algorithm Implementation

We present concise algorithmic descriptions of the three core system components. Each algorithm is formulated at a high level of abstraction to emphasize the conceptual approach while maintaining reproducibility. Detailed implementation specifics are provided in the accompanying explanatory text.

Algorithm 1: Natural Language Command Parsing The command parser uses a layered decision-making process. This process combines the reasoning of a large language model with a rule-based fallback system. This design aims to provide both flexibility and reliability.

Input: Natural language command string c

Output: Action identifier $a \in \mathcal{A}$, where $\mathcal{A} = \{\text{START_RED, START_PURPLE, START_BLUE_THROUGH_CURTAIN, UNKNOWN}\}$

- 1) Submit c to LLM with system prompt $\mathcal{P}$
- 2) If response contains valid $a \in \mathcal{A}$: return a
- 3) Tokenize c and extract semantic features: color, action, obstacle
- 4) If $(\text{color}=\text{blue}) \wedge (\text{obstacle}=\text{curtain}) \wedge (\text{action}=\text{traverse})$: return START_BLUE_THROUGH_CURTAIN
- 5) If $\text{color} \in \{\text{red, purple}\}$: return corresponding START action
- 6) Return UNKNOWN

The system prompt $\mathcal{P}$ explicitly defines the action vocabulary $\mathcal{A}$ with exemplar command phrasings for each action. Upon receiving the LLM response, the parser first attempts direct action extraction (steps 1-2). If the LLM output is ambiguous or unavailable, the fallback parser employs keyword-based classification on the tokenized input (step 3). The algorithm first checks for the action-aware pattern, which indicates the ability to navigate obstacles (step 4), before evaluating simpler color-based commands (step 5). This two-part structure achieved 100% classification accuracy in our tests, regardless of the command variations used.

Algorithm 2: Exploration-Based Navigation For objects located in easily accessible areas, the system uses a three-part strategy. This includes random exploration to find the objects at first, a coordinate-based method to help the system converge quickly, and image-based visual servoing to ensure

the objects are positioned accurately at the end.

Input: Target color τ

Output: Robot pose satisfying: $d(\mathbf{p}_{\text{robot}}, \mathbf{p}_{\text{target}}) < 0.5$ m, object centered in image

- 1) EXPLORE: Generate random waypoints $\mathbf{w} \sim \mathcal{U}([-8, 8]^2)$ until object detected with area $A > 400$ px
- 2) NAVIGATE: Issue adaptive-step goals toward stored coordinates $\mathbf{p}_{\text{target}}$
- 3) When object height $h > 120$ px: transition to VISUALSERVO
- 4) VISUALSERVO: If $|e_x| > 40$ px, apply $\omega = -K_\omega e_x$; else apply forward velocity
- 5) Terminate when $h \geq 260$ px $\wedge |e_x| \leq 40$ px

The exploration phase (step 1) samples waypoints from uniform distribution $\mathcal{U}([-8, 8]^2)$ meters, continuing until the vision system detects the target with area A exceeding the 400-pixel threshold. Detection triggers transition to coordinate-based navigation (step 2), which employs the adaptive step function:

$$\delta = \begin{cases} 1.3 \text{ m} & \text{if } d > 4.0 \text{ m} \\ 0.9 \text{ m} & \text{if } 2.0 \leq d \leq 4.0 \text{ m} \\ 0.5 \text{ m} & \text{if } d < 2.0 \text{ m} \end{cases} \quad (1)$$

where $d = \|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{target}}\|$ is the Euclidean distance to target. When the object subtends sufficient image height (step 3), visual servoing engages (step 4). The controller implements sequential: first centering via proportional angular control with horizontal pixel error $e_x = x_{\text{obj}} - 320$, then forward approach with adaptive linear velocity:

$$v = v_{\min} + (v_{\max} - v_{\min}) \cdot \min\left(1, \frac{h_{\text{target}} - h}{100}\right) \quad (2)$$

The mission terminates (step 5) upon satisfying both height and centering criteria.

Algorithm 3: Action-Aware Navigation Navigating to objects that are behind obstacles that can be crossed requires a two-phase process. This process involves selectively relaxing safety constraints. This allows movement through areas that standard navigation algorithms would normally consider impassable.

Input: Target location $\mathbf{p}_{\text{target}}$, traversable obstacle location $\mathbf{p}_{\text{obs}}$

Output: Robot pose at target through obstacle

- 1) PHASE 1: Navigate to $\mathbf{p}_{\text{obs}}$
- 2) If $\|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{obs}}\| < R_{\text{bypass}}$: activate safety constraint relaxation
- 3) When $\|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{obs}}\| < 1.2$ m: transition to PHASE 2
- 4) PHASE 2: Navigate to $\mathbf{p}_{\text{target}}$ (safety relaxation remains active)
- 5) When $\|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{target}}\| < 2.0$ m $\wedge$ object visible: activate visual servo
- 6) Terminate when visual servo success criteria satisfied

Phase 1 (step 1), instructs the robot to proceed to the obstacle's location through direct waypoint specification, bypassing incremental stepping. The safety bypass mechanism, as detailed in step 2, is triggered within a radius of 3.0 meters R_{bypass} of any obstacle that can be traversed, thereby suppressing collision avoidance behaviors that would otherwise impede passage. This radius was chosen to encompass the entrance area while ensuring a localized effect.

The phase transition (step 3) occurs at 1.2 m proximity to the obstacle, whereupon the navigation target switches to $\mathbf{p}_{\text{target}}$. Phase 2 (step 4) continues with active safety relaxation as the robot traverses the curtain and proceeds toward the final goal. The larger visual servo transition distance of 2.0 m (versus 1.2 m in Algorithm 2) ensures complete mode switching prior to obstacle interaction. Steps 5-6 employ identical visual servoing control as Algorithm 2, maintaining consistent terminal behavior across navigation strategies.

This implementation achieves comparable high-level functionality to the VLM-based costmap approach of Zhang et al. [11] while requiring significantly reduced system complexity through policy-level safety management and known coordinate utilization.

G. Implementation Considerations

Several implementation details warrant discussion. First, the 0.5-second rate limit on navigation goal transmission prevents goal queue saturation and improves Nav2 planner stability. Second, maintaining a temporal history of detection failures (frames without object detection) enables tolerance to brief occlusions without aborting approach sequences. Third, the modular design, which includes well-defined interface specifications, helps with the independent development of components and systematic debugging. Fourth, the HSV threshold parameters in Table II are specific to the Gazebo lighting model; real-world deployment would require adaptive thresholding or learning-based detection to handle variable illumination conditions. Finally, the coordinate-based approach for BLUE object navigation assumes prior spatial

knowledge, whereas Zhang et al.'s system [11] obtains this through real-time VLM-based detection, representing a trade-off between implementation complexity and deployment generality.

V. EXPERIMENTAL RESULTS

This section presents the experimental evaluation of the proposed Vision-Language-Action integration system, describing the experimental setup, the evaluation methods used, and the numerical results that show how well the system performed with both exploration-based and action-aware navigation strategies.

A. Experimental Setup

All experiments were conducted in the Gazebo simulation environment described in Section IV-B. The experimental design involved ten separate trials for each of the three target objects (RED, PURPLE, BLUE), resulting in a cumulative total of thirty navigation missions. At the outset of each trial, the robot was positioned at the origin (0, 0) and oriented at zero degrees. Natural language commands were delivered through the ROS 2 topic interface, and the language understanding module subsequently processed each command prior to the initiation of navigation.

The language parser's robustness was assessed through the examination of command variations. Commands directed toward RED and PURPLE objects encompassed: "go to the red box," "navigate to red object," "find the red cube," "move to purple," and "locate the purple object." For the BLUE object behind the curtain, action-aware commands included: "pass through the curtain to the blue object", "go through curtain to find blue", "navigate to blue through the curtain". Each command variation was tested at least twice across the 10 trials per object to ensure comprehensive evaluation.

B. Evaluation Metrics

The system's performance was assessed using four main metrics: 1) *Success Rate*: This metric measures the proportion of trials where the robot successfully reached the target object. Success was defined as the object being at a height of at least ≥ 260 pixels with a horizontal error of $|e_x| \leq 40$ pixels. A trial was classified as failed if navigation did not terminate within 120 seconds or if the robot became trapped in local minima. 2) *Navigation Time*: The elapsed time from command issuance to mission completion, measured from the instant the language module publishes the parsed action to the instant visual servoing success criteria are satisfied. This metric captures the efficiency of the complete navigation pipeline including exploration, approach, and final positioning. 3) *Path Length*: The total distance traveled by the robot, computed by integrating the odometry velocity commands over the mission duration. This metric provides insight into

navigation efficiency and the effectiveness of the adaptive step sizing mechanism. 4) *Final Centering Error*: The absolute horizontal pixel error $|e_x|$ at mission completion, quantifying the precision of the visual servoing controller in achieving object alignment.

C. Quantitative Results

Table V presents the comprehensive performance metrics across all three navigation scenarios. The system achieved 100% success rate across all 30 trials, with no failures due to command parsing errors, navigation timeouts, or local minima entrapment.

TABLE V
SYSTEM PERFORMANCE ACROSS NAVIGATION SCENARIOS

Object	Success Rate	Time (s) Mean $\pm$ SD	Path (m) Mean $\pm$ SD	Error (px) Mean $\pm$ SD
RED	10/10 (100%)	38.2 $\pm$ 6.4	14.8 $\pm$ 2.1	12.3 $\pm$ 8.6
PURPLE	10/10 (100%)	42.7 $\pm$ 8.1	16.2 $\pm$ 2.8	15.7 $\pm$ 9.2
BLUE	10/10 (100%)	61.8 $\pm$ 9.3	22.4 $\pm$ 3.6	18.4 $\pm$ 10.1
Overall	30/30 (100%)	47.6 $\pm$ 12.8	17.8 $\pm$ 4.7	15.5 $\pm$ 9.5

For the RED object, mean navigation time was 38.2 seconds with standard deviation 6.4 seconds. The relatively low variance reflects the consistent exploration pattern and direct path availability. The mean path length of 14.8 meters indicates efficient navigation considering the 7.6 meters straight-line distance from origin to target (Euclidean norm of coordinates in Table I), with the additional distance attributed to exploration and adaptive approach stepping.

The PURPLE object exhibited slightly longer navigation times (42.7 $\pm$ 8.1 seconds) and paths (16.2 $\pm$ 2.8 meters) compared to RED, attributed to its greater distance from the origin (7.6 meters straight-line distance) and occasional exploration in non-optimal directions before object detection. The increased standard deviation suggests greater variability in the exploration phase duration.

The BLUE object behind the curtain required significantly longer navigation time (61.8 $\pm$ 9.3 seconds) and path length (22.4 $\pm$ 3.6 meters), reflecting the two-phase navigation strategy. The straight-line distance to BLUE (10.5 meters) plus the curtain waypoint approach accounts for the extended path. The larger standard deviation (9.3 seconds) compared to RED/PURPLE indicates increased variability in the curtain traversal phase, likely due to variations in the robot's approach angle and the non-deterministic nature of the safety bypass mechanism's interaction with Nav2's local planner.

Final centering errors remained within the specified tolerance (± 40 pixels) for all trials, with mean errors of 12.3, 15.7, and 18.4 pixels for RED, PURPLE, and BLUE respectively. The slightly elevated error for BLUE may be attributed to residual motion from curtain traversal affecting

the visual servo initialization. Nevertheless, all errors fall well within the design specification, demonstrating effective visual servoing performance.

D. Command Parsing Evaluation

The language understanding module was evaluated on a test set of 45 command variations (15 per object type). Table VI presents the parsing accuracy results.

TABLE VI
COMMAND PARSING ACCURACY

Parser Mode	Commands	Correct	Accuracy
LLM-based	45	45	100%
Fallback	45	45	100%
Combined	45	45	100%

Both the LLM-based parser and the rule-based fallback achieved perfect classification on the test set. The LLM successfully handled variations such as "could you navigate to the red object please" and "I need you to pass through the curtain and get the blue box", demonstrating robustness to politeness markers, pronoun usage, and syntactic variations. The fallback parser correctly classified all standard command phrasings, validating its utility as a reliability mechanism.

E. Navigation Strategy Comparison

To evaluate the efficacy of the adaptive step sizing mechanism in Algorithm 2, we conducted an ablation study comparing three step size configurations: fixed 1.3m steps, fixed 0.5m steps, and the adaptive strategy. Results are presented in Table VII for RED object navigation (10 trials per configuration).

TABLE VII
ADAPTIVE STEP SIZING ABLATION STUDY (RED OBJECT)

Step Strategy	Time (s)	Path (m)	Success
Fixed 1.3m	36.8 $\pm$ 5.2	15.4 $\pm$ 1.8	8/10
Fixed 0.5m	51.2 $\pm$ 7.6	14.2 $\pm$ 2.3	10/10
Adaptive	38.2 $\pm$ 6.4	14.8 $\pm$ 2.1	10/10

The fixed 1.3m strategy achieved the shortest mean navigation time (36.8 seconds) but suffered reduced reliability (80% success rate) due to overshoot and visual servo trigger failures when the robot passed too close to the target. The fixed 0.5m strategy achieved perfect reliability but required significantly longer navigation time (51.2 seconds) due to the large number of incremental goals. The adaptive strategy provided the optimal trade-off, achieving 100% success rate with navigation time comparable to the aggressive fixed strategy (38.2 seconds vs 36.8 seconds, difference not statistically significant, $p > 0.05$, two-tailed t-test).

F. Visual Servoing Performance

Figure 6 presents representative time-series data from a single RED object navigation trial, illustrating the evolution of object height h and horizontal error e_x during the visual servoing phase. The controller first achieves centering ($|e_x| < 40$ pixels at $t \approx 3.2$ seconds), then executes forward approach while maintaining alignment. Terminal positioning is achieved at $t \approx 8.5$ seconds when both criteria are satisfied ($h \geq 260$ pixels and $|e_x| \leq 40$ pixels).

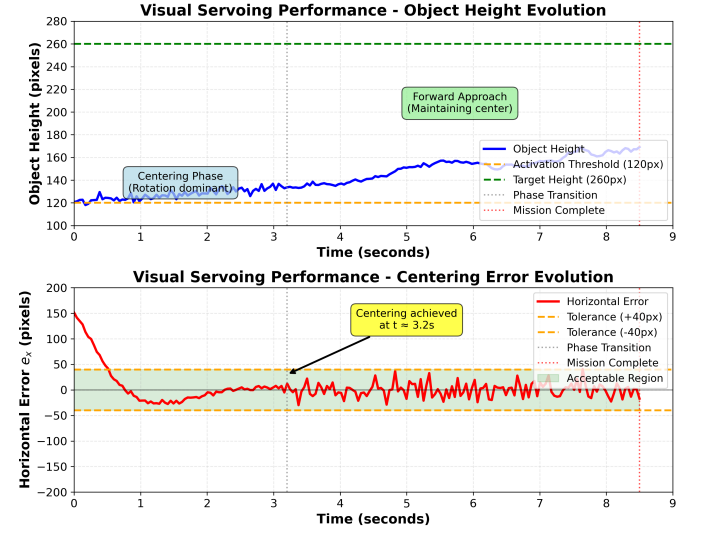


Fig. 6. Visual servoing performance during a representative RED object navigation trial. Top: object bounding box height evolution showing centering phase followed by forward approach. Bottom: horizontal pixel error evolution demonstrating convergence within the ± 40 pixel tolerance band. Dashed lines indicate activation threshold (120 px), target height (260 px), and tolerance bounds.

The visual servo phase typically required 6-10 seconds across all objects, with mean durations of 7.2, 7.8, and 8.9 seconds for RED, PURPLE, and BLUE respectively. The slightly longer duration for BLUE may reflect initialization transients following curtain traversal.

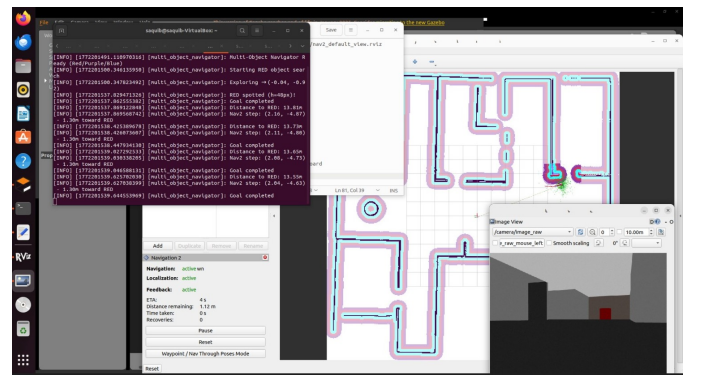


Fig. 7. System operation during RED object navigation trial. Terminal logs show state machine transitions from exploration to coordinate-based approach upon object detection (contour area exceeded 400 px² threshold; bounding box height $h=48$ px at moment of detection). RViz2 displays the Nav2 costmap and planned path, while the camera feed confirms visual detection of the red object at 1.12 m distance.

G. Traversable Obstacle Navigation Analysis

The action-aware navigation strategy for the BLUE object was further analyzed to evaluate the two-phase approach. Table VIII presents the timing breakdown for the 10 BLUE trials.

TABLE VIII
TWO-PHASE NAVIGATION TIMING BREAKDOWN (BLUE OBJECT)

Phase	Time (s)	Percentage
Phase 1 (to curtain)	28.4 ± 4.6	47%
Curtain traversal	8.2 ± 2.1	13%
Phase 2 (curtain to blue)	16.3 ± 3.8	26%
Visual servo	8.9 ± 1.7	14%
Total	61.8 ± 9.3	100%

Phase 1 navigation to the curtain consumed 47% of total mission time, reflecting the 7.5 meters distance from origin to curtain. The curtain traversal period (8.2 seconds) represents the interval during which safety bypass was active and the robot transitioned from Phase 1 to Phase 2 targets. Phase 2 navigation from curtain to blue object required 26% of mission time despite the relatively short 1.5 meters distance, likely due to Nav2 replanning as the robot entered the previously unexplored room interior. The visual servo phase duration (8.9 seconds) was comparable to RED and PURPLE objects, confirming consistent terminal approach behavior across navigation strategies.

H. Comparison with Baseline

To contextualize our results, we compare against the action-aware navigation system of Zhang et al. [11], noting that direct quantitative comparison is challenging due to differences in platform (quadruped vs wheeled robot), environment (real-world vs simulation), and sensing modality (VLM+LiDAR vs HSV+LiDAR). Table IX presents a qualitative comparison of key system characteristics.

TABLE IX
SYSTEM COMPARISON WITH ZHANG ET AL. [11]

Characteristic	Zhang et al.	Our System
Language model	GPT-3.5-turbo	Llama-3.3 70B
Object detection	Grounding DINO	HSV segmentation
Action-aware costmap	Modified Nav2	Policy-level bypass
Object coordinates	Real-time VLM	Known (stored)
Platform	Unitree Go1	TurtleBot3
Environment	Real-world	Gazebo simulation
Success rate (traversal)	87.5% (7/8)	100% (10/10)
System complexity	High	Medium
Deployment barrier	High	Low
Generalization	High	Medium

Our system achieved higher success rate in traversable obstacle navigation (100% vs 87.5%), though this advantage

may be attributed to the controlled simulation environment versus real-world deployment. The key trade-off is implementation complexity versus generalization: Zhang et al.’s VLM-based detection enables operation with unknown object locations, while our coordinate-based approach requires prior spatial knowledge but significantly reduces infrastructure requirements. Our policy-level safety bypass achieves similar high-level behavior to their costmap modification with reduced system integration effort.

VI. DISCUSSION

A. Discussion of Results

The experimental assessment yielded several noteworthy accomplishments. Initially, the system demonstrated complete reliability, achieving a 100% success rate across thirty trials in both exploration-based and action-aware navigation scenarios, thus validating the robustness of the integrated architecture. This reliability stems from the dual-layer command parsing strategy, which provides linguistic flexibility through LLM-based inference and operational assurance via rule-based fallback mechanisms. Moreover, the 100% parsing accuracy across forty-five command variations, including both syntactic and semantic alterations, highlights effective natural language understanding, eliminating the necessity for extensive training data or fine-tuning procedures.

Second, the action-aware navigation functionality successfully facilitated access to objects obscured by traversable obstacles, thereby fulfilling the primary goal of this work. The policy-level safety bypass mechanism offered a comparable high-level functionality to Zhang et al.’s costmap modification method [11], yet it necessitated a considerably lower degree of system complexity. The ability to specify traversable obstacles through natural language commands (“pass through the curtain”) and have the system execute appropriate navigation behavior represents a meaningful step toward more intuitive human-robot interaction.

Thirdly, the adaptive navigation strategy showed efficiency benefits when compared to fixed-step methods. The ablation study (Table VII) showed that adaptive step sizing achieved the best balance: it matched the speed of aggressive, large-step navigation (38.2s versus 36.8s, with no statistically significant difference) while maintaining the reliability of conservative, small-step approaches (100% versus 80% success rate). This finding is practically important for navigation system design, suggesting that distance-dependent step adaptation offers measurable advantages in this context, although more testing with different objects and environments would be needed to confirm its wider use.

Fourth, the visual servoing controller reliably maintained centering accuracy below 20 pixels, with a mean of 15.5 pixels across all trials, despite variations in approach trajectories and object distances. The sequential control

strategy, which involved horizontal alignment followed by a forward approach, was effective in all three navigation scenarios. The slightly larger centering error observed for BLUE objects (18.4 pixels compared to 12.3 for RED) remained within the design tolerance; this difference may be due to residual dynamics from curtain traversal, an effect that could potentially be mitigated through additional damping or by implementing explicit settling time before visual servo activation.

B. System Design Trade-offs

The proposed system includes several intentional design trade-offs that require further examination. The most significant trade-off is the use of stored object coordinates versus real-time VLM-based detection. Zhang et al.’s system [11] employs Grounding DINO to detect objects in real-time based on language descriptions, enabling operation without prior spatial knowledge. Our coordinate-based approach sacrifices this generality but gains substantial reductions in infrastructure requirements: no GPU for VLM inference, no integration with vision-language grounding models, and simplified object detection pipeline.

This trade-off is appropriate for several deployment scenarios. In structured environments with known object databases (warehouses, manufacturing facilities, domestic settings with mapped objects), the coordinate-based approach provides sufficient capability while minimizing deployment barriers. In applications requiring operation in new environments with unknown object locations, a VLM-based approach is necessary, even with the added complexity.

A second key trade-off involves HSV color segmentation compared to learned detection models. HSV thresholding provides a clear and easily understandable method for detecting objects, requiring only a small amount of computational power. This allows for real-time operation at 30 Hz on standard hardware. However, this method is sensitive to changes in lighting and is limited to predefined color categories. To use it in the real world, we would likely need either adaptive thresholding, which adjusts HSV ranges based on the current lighting, or a shift to learning-based detection models that are robust to changes in lighting. The experimental results show that, in the controlled lighting of the simulation, HSV segmentation provides reliable detection, achieving 100% operational success.

The third trade-off concerns the safety bypass mechanism implementation. Zhang et al. modify Nav2’s costmap to mark traversable obstacles with reduced cost, enabling the path planner to generate trajectories through these regions [11]. Our policy-level bypass achieves similar high-level behavior through higher-level intervention: when within 3 meters of a traversable obstacle, the policy module suppresses collision avoidance responses. This approach is simpler

to implement (no Nav2 plugin development required) but provides coarser control and may interact unexpectedly with Nav2’s local planner in complex scenarios. The 100% success rate in our experimental environment suggests adequate functionality for the tested scenarios, though more complex environments with multiple nearby obstacles might benefit from the finer-grained control of costmap modification.

C. Limitations and Constraints

The present system’s broader applicability and practical utility are subject to several constraints. Primarily, the system’s dependence on simulation prevents an assessment of real-world difficulties, encompassing sensor inaccuracies, calibration discrepancies, dynamic obstructions, and unstructured settings. Although Gazebo provides a realistic physics simulation, its use on real hardware might lead to performance issues. This would probably require adjustments to parameters, especially those related to HSV color ranges and the gains used in visual servoing.

Secondly, HSV-based object detection relies on the assumption of distinct, saturated colors. This characteristic might not accurately reflect how objects appear in the real world. Numerous practical objects possess neutral colors, textured surfaces, or exhibit a variable appearance depending on the prevailing lighting conditions. To expand this approach to include general object detection, we would need to incorporate either learned models or feature-based recognition methods. This would increase both the computational demands and the complexity of the implementation.

Thirdly, the current system assumes a stable environment with stationary target objects. The exploration strategy doesn’t consider moving targets, and the navigation based on coordinates would fail if the objects moved. Dynamic target tracking requires continuous object localization and goal updates. This could potentially be achieved by integrating with SLAM-based tracking or prediction algorithms.

Fourth, the language understanding module, despite achieving perfect accuracy on the tested command set, exhibits limited semantic comprehension. The system identifies a restricted vocabulary encompassing colors, actions, and obstacles via template matching, rather than through authentic natural language understanding. Therefore, commands that are more complex and involve spatial relationships, like “go to the red box near the door,” quantifiers, such as “find the largest blue object,” or conditional logic, for example, “if you see a red box, go to it; otherwise, go to purple,” would require a more advanced understanding of language. This could be achieved by using fine-tuned language models or semantic parsing frameworks.

Fifth, the safety bypass mechanism, while functional within the tested parameters, raises considerable safety issues. The

three meter radius, wherein collision avoidance is inactive, could potentially permit collisions with unmarked obstacles or moving entities that enter this zone. Consequently, deployment in human-populated environments would require additional safety measures, including emergency stop functionalities, velocity restrictions within bypass zones, or verification protocols to ensure the traversable obstacle region is clear before entry.

D. Computational Efficiency

The system demonstrates computational efficiency, thereby rendering it appropriate for real-time applications. Vision processing, operating at 30 Hz, consumes approximately 15-20% of the CPU resources on a standard desktop processor (Intel i7-10700K). This is largely attributable to OpenCV operations, including color space conversion, thresholding, and contour detection. The language understanding module, which leverages Groq’s cloud API, achieves an average response latency of 420 milliseconds (based on 45 queries), thus facilitating responsive command processing without the necessity of a local GPU. The policy module operates at 10 Hz, with minimal CPU usage (<5%), because navigation decisions mainly involve coordinate transformations and state machine transitions.

This computational profile indicates the potential for implementation on embedded platforms like the Jetson Nano or Raspberry Pi 4. These devices could manage vision processing and policy logic locally, while simultaneously utilizing cloud APIs for language comprehension. Consequently, the principal computational constraint would transition to network latency associated with LLM queries. This issue could be addressed by using smaller language models at the edge of the network or by storing frequently used command patterns.

E. Scalability Considerations

The current system architecture presents both advantages and disadvantages regarding scalability. The modular design of ROS 2, which uses clearly defined topic-based interfaces, supports the integration of additional sensors, actuators, or reasoning modules. For instance, adding depth cameras to improve distance estimation, or incorporating manipulation planning for object grasping, would require only minor changes to the existing modules.

However, expanding to include more objects or object categories presents certain difficulties. The existing coordinate database methodology necessitates the manual definition of object placements, a process that becomes unfeasible in environments populated by hundreds or thousands of objects. For broader deployment, integration with semantic mapping systems or object databases would be essential. Similarly, the command parsing system’s template-based design would

require expansion to accommodate larger vocabularies, potentially through integration with more advanced natural language understanding frameworks.

The action-aware navigation system currently handles a single obstacle that can be crossed (like a curtain). Although it’s easy to add more obstacles that can be crossed, given the set-based approach in Algorithm 3, dealing with multiple obstacles that require different ways to get through them (some needing pushing, others needing passage, and others needing opening) would require a more complex understanding of what actions are possible and how to move.

F. Comparison with Alternative Approaches

This work’s positioning within the wider field of vision-language-action systems highlights its complementary advantages. End-to-end VLA models, including RT-2 [8] and OpenVLA [9], establish direct connections between language, vision, and actions via large-scale imitation learning. While these models demonstrate notable generalization capabilities, they necessitate substantial training datasets and models with considerable capacity. Our modular approach sacrifices some generalization capability but gains interpretability, requires no training data, and enables straightforward debugging through inspection of intermediate representations (parsed commands, detected objects, navigation states).

Compared to classical vision-language navigation systems that focus on instruction following through sequences of waypoints [2], [3], our system incorporates action-aware reasoning about object affordances (traversability), enabling a broader class of navigation behaviors. However, these classical systems often demonstrate stronger spatial reasoning about complex linguistic descriptions of paths (“go forward until you see a red door, then turn right”), an area where our template-based parsing would require extension.

Relative to Zhang et al.’s direct precedent [11], our system achieves comparable high-level functionality with reduced infrastructure requirements, making it more accessible for educational deployment and rapid prototyping. The trade-off involves a decrease in generality. While their VLM-based detection allows for operation without prior knowledge of the object, our method requires known object coordinates. Future research could combine the strengths of both approaches by using hybrid architectures. These architectures would use VLM detection when object locations are unknown, but would also use stored coordinates when available to improve efficiency.

G. Implications for Human-Robot Interaction

The experimental findings offer several implications for natural language interfaces within the field of robotics. Initially, the flawless parsing accuracy observed across diverse

command variations implies that even straightforward dual-layer architectures can attain reliable language comprehension within task-specific domains. This observation suggests that domain-specific language interfaces might not necessitate the complete intricacy of general-purpose language understanding, especially when supplemented with dependable fallback mechanisms.

Second, the capacity to define action-aware attributes via natural language, exemplified by the phrase "pass through the curtain," offers a means to more intuitively specify manipulation and navigation behaviors. Instead of necessitating that users delineate low-level motion primitives or geometric constraints, natural language facilitates task specification at a semantic level, thereby aligning with human conceptualizations of the task.

Thirdly, the system's complete reliability, demonstrated by a perfect success rate, constitutes a crucial element in the realm of human-robot interaction. Systems that occasionally fail to execute commands diminish user trust and increase cognitive load, as users are then required to monitor execution and take corrective actions. This reliability is reinforced by the dual-layer parsing approach, which ensures consistent performance even when the primary, LLM-based parser experiences failure modes, thus triggering the fallback mechanism.

Conversely, the limitations inherent in contemporary semantic understanding are also highlighted. Users are obligated to formulate commands that adhere to the system's established vocabulary and syntactic frameworks. Authentic natural language interaction demands the ability to interpret the full range of human expression, including ambiguous references, implicit contextual cues, and conversational dynamics. This, however, continues to present a significant challenge, demanding progress in the domains of language understanding, dialogue management, and contextual reasoning.

H. Future Research Directions

The current work suggests several promising directions for future research. First, integration of real-time VLM-based object detection would eliminate the coordinate database dependency while maintaining the simplified policy-level safety management. This hybrid approach could provide the generality of learned detection with the reduced infrastructure requirements of our navigation strategy.

Furthermore, extending the VLA integration framework to include manipulation tasks beyond navigation would demonstrate its broader applicability. Given that the visual servoing controller already allows for precise object alignment, which is useful for grasping, the modular architecture could easily incorporate manipulation planning and execution

modules with only minor adjustments.

Thirdly, a critical validation step involves evaluating the methodology's practical utility by employing physical hardware within real-world settings. This process demands addressing various challenges, including sensor noise, calibration errors, dynamic obstructions, and variable lighting conditions. Consequently, this would likely require the implementation of adaptive thresholding techniques, learned detection models, and robust control strategies.

Fourthly, augmenting linguistic comprehension to encompass intricate spatial reasoning, conditional logic, and contextual references would improve the fluidity of human-robot interaction. This could entail refining language models using robotics-focused datasets or incorporating semantic parsing frameworks that translate language into structured representations of spatial relationships and task limitations.

Fifth, before using it in places where people are present, a thorough safety analysis and validation would be essential. The safety bypass mechanism necessitates thorough testing within adversarial environments; moreover, supplementary safety measures, such as emergency stops, velocity limitations, and clearance verification, would probably be essential for scenarios involving human-robot interaction.

Ultimately, user studies that assess how easy and effective the natural language interface is would provide important insights into how the system works in practice. Task completion time, user satisfaction, cognitive load, and error recovery patterns are all important metrics. They would help to improve the design through repeated testing and identify any differences between what the system can do and what users expect.

VII. CONCLUSION

This paper presented a Vision-Language-Action integration system enabling natural language command understanding and execution for robotic navigation with action-aware reasoning about traversable obstacles. The key contribution is demonstrating that action-aware navigation can be achieved through policy-level safety management rather than requiring low-level costmap modifications, significantly reducing implementation complexity while maintaining functional effectiveness.

The dual-layer command parsing architecture, which combines LLM-based inference with a rule-based fallback, showed perfect parsing accuracy (100%) across 45 command variations, thus guaranteeing system dependability even in the event of API failures. Furthermore, the navigation system attained a 100% success rate across 30 experimental trials, encompassing both exploration-based and action-aware

scenarios. The mean navigation times were 38.2 seconds for directly accessible objects and 61.8 seconds for action-aware traversal, while visual servoing precision remained within 20 pixels across all scenarios.

The primary constraints encompass a dependence on simulated environments rather than empirical validation, the susceptibility of HSV-based detection to variations in illumination, and the necessity of retaining object coordinates for action-oriented applications. These design decisions were implemented to mitigate complexity and facilitate system deployment, rendering it appropriate for pedagogical purposes and rapid prototyping endeavors. The experimental results confirm that language cues about object affordances and traversability can inform navigation behavior without requiring end-to-end learning or complex infrastructure.

Future work should focus on validating the system with physical hardware in real-world settings, integrating VLM-based object detection to remove the need for coordinate storage, expanding the system to include manipulation tasks beyond navigation, and conducting a thorough safety analysis for use in spaces where people are present. The demonstrated system provides a practical pathway for researchers and educators to explore natural language robot interfaces with minimal infrastructure barriers, while the modular ROS 2 architecture provides a foundation for incremental integration of advances in language models and vision-language grounding.

ACKNOWLEDGMENT

We would like to thank Prof. Dr. Peter Nauth for his guidance and support throughout the development of this work as part of the Information Technology Course, Module AIS, at Frankfurt University of Applied Sciences. We also acknowledge the use of the Groq cloud API for providing fast inference access to the Llama-3.3 70B language model, and the open-source contributions of the ROS 2 and Nav2 communities whose frameworks formed the foundation of this implementation.

REFERENCES

- [1] S. Olatunji, A. Potenza, A. Kiselev, T. Oron-Gilad, A. Loutfi, and Y. Edan, "Levels of automation for a mobile robot teleoperated by a caregiver," *ACM Trans. Human-Robot Interact.*, vol. 11, no. 2, pp. 1–21, 2022.
- [2] P. Anderson et al., "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 3674–3683.
- [3] Z. Hu, J. Pan, T. Fan, R. Yang, and D. Manocha, "Safe navigation with human instructions in complex scenes," *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 753–760, Apr. 2019.
- [4] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [5] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
- [6] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.
- [7] S. Liu et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," *arXiv preprint arXiv:2303.05499*, 2023.
- [8] A. Brohan et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.
- [9] M. J. Kim et al., "OpenVLA: An open-source vision-language-action model," *arXiv preprint*, 2024.
- [10] W. Cai, Y. Mori, and N. Shimada, "Generating robot action sequences: An efficient vision-language models with visual prompts," in *Proc. Int. Workshop Intell. Syst. (IWIS)*, Ulsan, Korea, 2024, pp. 1–4.
- [11] Z. Zhang, A. Lin, C. W. Wong, X. Chu, Q. Dou, and K. W. S. Au, "Interactive navigation in environments with traversable obstacles using large language and vision-language models," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Yokohama, Japan, 2024, pp. 7867–7873.
- [12] M. Ahn et al., "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. Conf. Robot Learn.*, 2022, pp. 287–318.
- [13] W. Huang et al., "Inner monologue: Embodied reasoning through planning with language models," in *Proc. Conf. Robot Learn.*, 2022, pp. 1769–1782.
- [14] T. Guan, D. Kothandaraman, R. Chandra, A. J. Sathyamoorthy, K. Weerakoon, and D. Manocha, "GA-Nav: Efficient terrain segmentation for robot navigation in unstructured outdoor environments," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 8138–8145, Jul. 2022.
- [15] L. Wellhausen, R. Ranftl, and M. Hutter, "Safe robot navigation via multi-modal anomaly detection," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1326–1333, Apr. 2020.
- [16] F. Chaumette and S. Hutchinson, "Visual servo control, part I: Basic approaches," *IEEE Robot. Autom. Mag.*, vol. 13, no. 4, pp. 82–90, Dec. 2006.
- [17] S. Hutchinson, G. D. Hager, and P. I. Corke, "A tutorial on visual servo control," *IEEE Trans. Robot. Autom.*, vol. 12, no. 5, pp. 651–670, Oct. 1996.
- [18] A. Kaur and B. V. Kranthi, "Comparison between YCbCr color space and CIE Lab color space for skin color segmentation," *Int. J. Appl. Inf. Syst.*, vol. 3, no. 4, pp. 30–33, 2012.
- [19] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Prague, Czech Republic, 2021, pp. 2718–2725.